Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра программного обеспечения информационных технологий

Специальность «Программная инженерия»

Учебная дисциплина «Метрология, стандартизация и сертификация в информационных технологиях»

ОТЧЕТ

по практической работе №2б
«Метрика граничных значений»

Подготовили: Гавлев Н.В.
Ст.гр.410902 Джалилова Н.Р.

Проверил: Болтак С.В.

Минск 2026

Язык для анализа: Rust

Задание: На основании составленной граф-схемы алгоритма рассчитать абсолютную Sa и относительную So граничную сложность программы по метрике граничных значений.

Для расчета метрики необходимо изобразить код анализируемой программы в виде подробной схемы алгоритма по ГОСТ 19.701-90. Это означает, что каждому блоку схемы алгоритма должен соответствовать один оператор языка программирования. Например, ввод или вывод двумерного массива должен быть представлен не одним блоком ввода/вывода (параллелограмм), а двумя циклами. Каждый цикл изображается с обязательной инициализацией и изменением переменной цикла. Оператор множественного выбора изображается в виде вложенных друг в друга операторов условия. Для переноса блоков на другую страницу использовать коннекторы.

Листинг анализируемого кода:

```
fn calculate_score(values: &[i32]) -> i32 {
    let mut score = 0;
    let mut index = 0usize;
    while index < values.len() {
        let value = values[index];
        if value != 0 {
            if value > 0 {
                score += value;
            } else {
                score -= value.abs();
            }
        } else {
            score += 1;
        }
        index += 1;
    }
    score
}
fn classify(value: i32) -> &'static str {
    match value {
        0 => "zero",
        1 => "one",
        2 => "two",
        3 => "three",
        _ => "other",
    }
}
fn process(values: &[i32]) -> i32 {
    let mut total = 0;
```

```
    let mut index = 0usize;
    while index < values.len() {
        let value = values[index];
        if value % 2 == 0 {
            let counter = 0usize;
            match counter {
                0 => total += 0,
                1 => {
                    if value > 10 {
                        total += 10;
                    } else {
                        total += 1;
                    }
                }
                _ => total += 0,
            }
        }
        index += 1;
    }
    total
}
fn main() {
    let values = vec![0, 1, 2, 5, 10, 15];

    let score = calculate_score(&values);
    let result = process(&values);

    println!("Score: {}", score);
    println!("Result: {}", result);

    let mut index = 0usize;

    while index < values.len() {
        let value = &values[index];

        if *value >= 0 {
            if *value > 10 {
                println!("{} -> large", value);
            } else {
                println!("{} -> {}", value, classify(*value));
            }
        } else {
            println!("{} -> negative", value);
        }
```

```
        index += 1;
    }
    println!("Score calculated: {}", score);
}
```

Схема алгоритма программы по ГОСТ19.701-90 представлена на рисунках 1, 2, 3, 4.

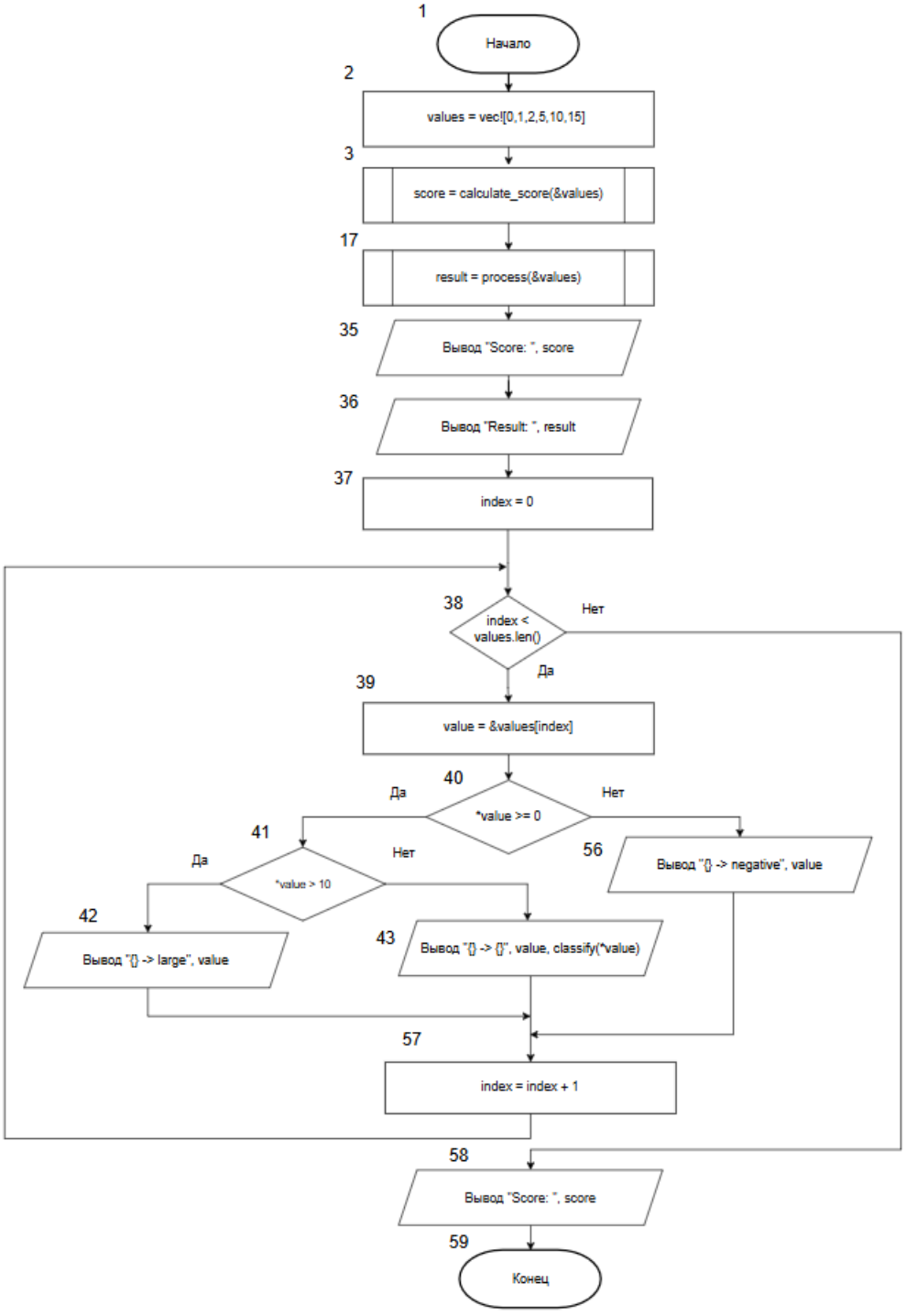


Рисунок 1 – Схема алгоритма программы (функция main)

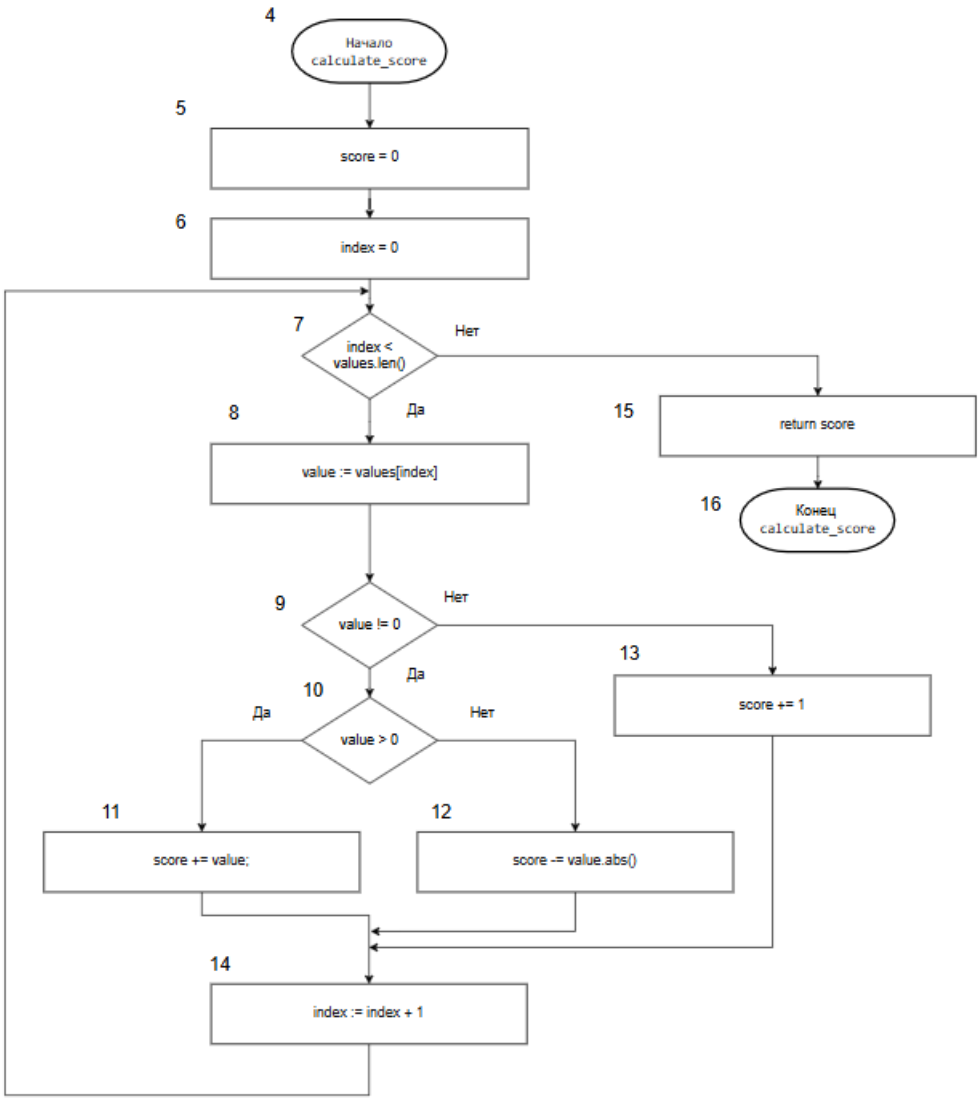


Рисунок 2 – Схема алгоритма программы (функция calculate_score)

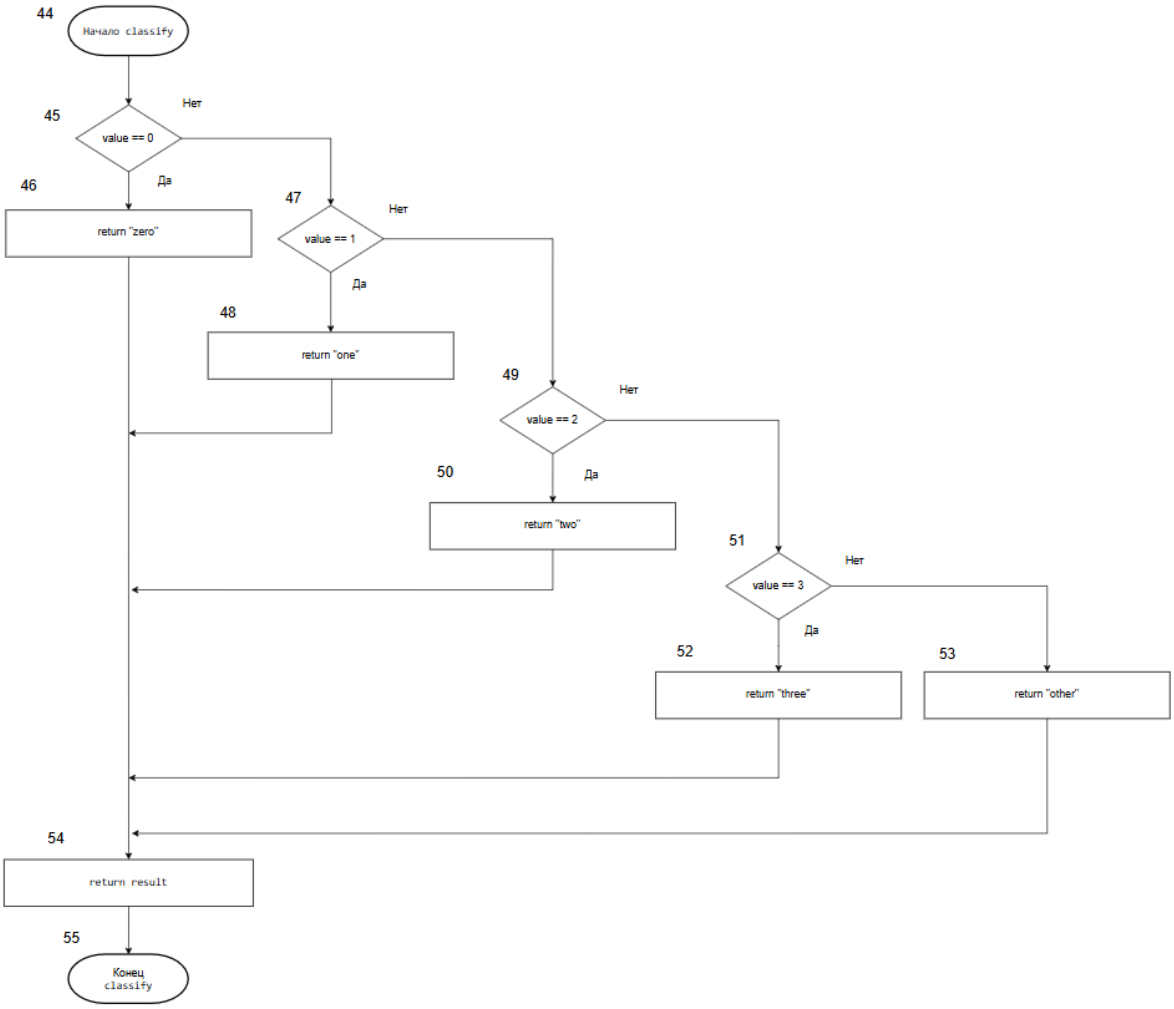


Рисунок 3 – Схема алгоритма программы (функция classify)

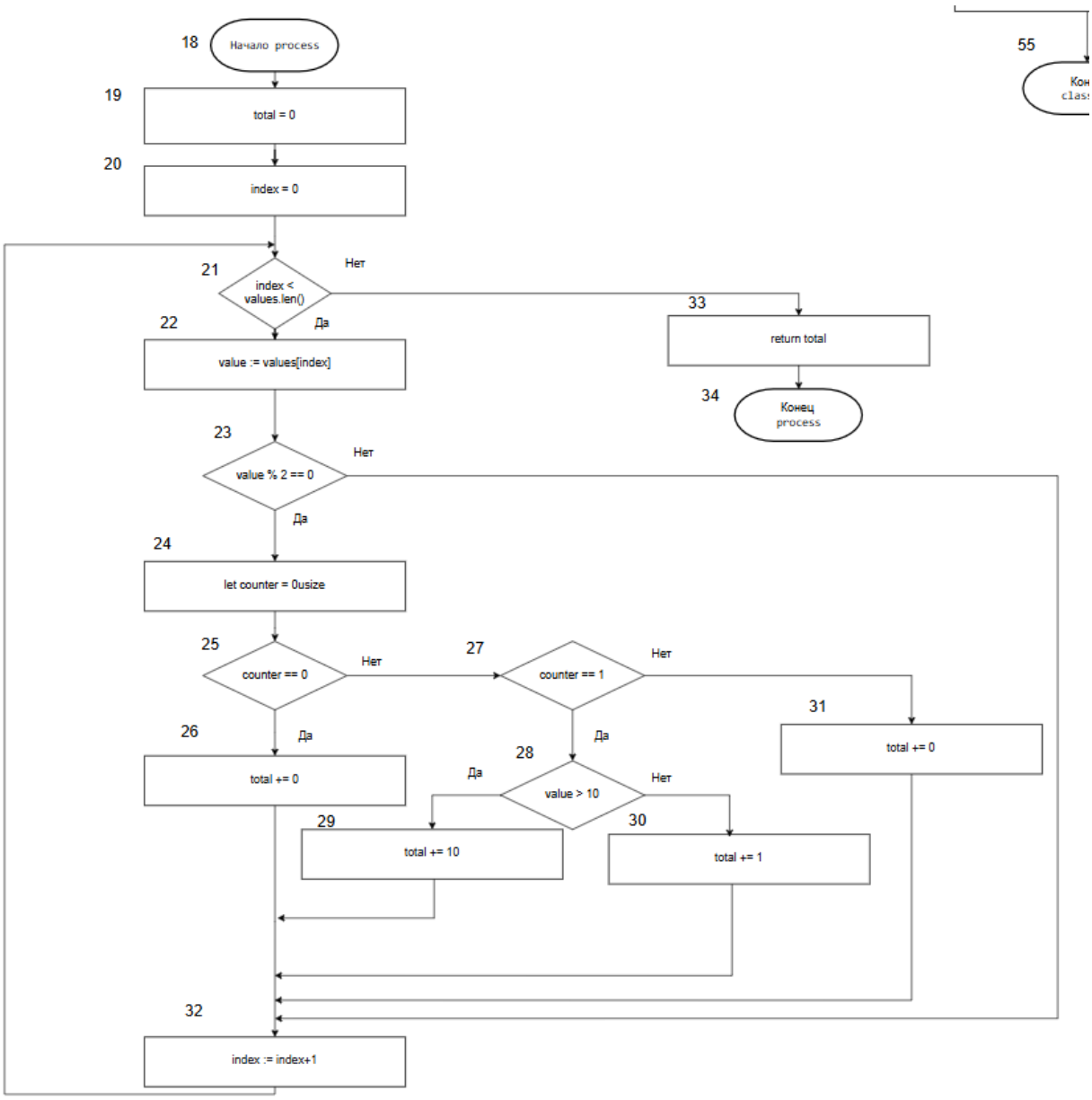


Рисунок 4 – Схема алгоритма программы (функция process)

В таблице 1 представлены свойства подграфов программы, таблица 2 содержит скорректированные сложности вершин графа данной программы.

Таблица 1– Свойства подграфов программы.

Номер вершины выбора	Свойства подграфов программы			
	Номера вершин перехода	Скорректированная сложность вершины выбора	Номера вершин подграфа	Номер нижней границы подграфа
calculate score ()				
7	8, 16	8	8, 9, 10, 11, 12, 13, 14	15
9	10, 13	6	10, 11, 12, 13,14	15
10	11, 12	4	11, 12, 14	15
process ()				
21	22, 33	12	22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32	33
23	24, 32	10	24, 25, 26, 27, 28, 29, 30, 31, 32	33
25	26, 27	8	26, 27, 28, 29, 30, 31, 32	33
27	31, 28	6	28, 29, 30, 31, 32	33
28	29, 30	4	29, 30, 32	33
main()				
38	39, 58	8	39, 40, 41, 42, 43, 56, 57	58
40	41, 56	6	41, 42, 43, 56, 57	58
41	42, 43	4	42, 43, 57	58
classify ()				
45	46, 47	9	46, 47, 48, 49, 50, 51, 52, 53	54
47	48, 49	7	48, 49, 50, 51, 52, 53	54
49	50, 51	5	50, 51, 52, 53	54
51	52, 53	3	52, 53	54

Таблица 2 – Скорректированные сложности вершин графа программы.

Номер вершины графа программы	Скорректированная сложность вершины графа	Номер вершины графа программы	Скорректированная сложность вершины графа
1	1	31	1
2	1	32	1
3	1	33	1
4	1	34	0
5	1	35	1
6	1	36	1
7	8	37	1
8	1	38	8
9	6	39	1
10	4	40	6
11	1	41	4
12	1	42	1
13	1	43	1
14	1	44	1
15	1	45	9
16	0	46	1
17	1	47	7
18	1	48	1
19	1	49	5
20	1	50	1
21	12	51	3
22	1	52	1
23	10	53	1
24	1	54	1
25	8	55	0
26	1	56	1
27	6	57	1
28	4	58	1
29	1	59	0
30	1		S a =140

Таким образом, абсолютная граничная сложность S a программы, схема алгоритма которой приведена на рис. 1, 2, 3, 4 равна 140.

Относительная граничная сложность данной программы равна
S o = 1 – (59 – 1)/140 = 0,586.